

+

New

AI Chat

Home

Skills

Claw

Workflows

Drive

...

More

Engine 1 crypto trading system code review request

Share

config.py4.84 KB

agent5_exact.py11.85 KB

unified_backtest.py8.39 KB

alpha_squeezer.py20.14 KB

Engine_1.py85.93 KB

Context

I am running a live crypto trading engine called Engine 1 on Binance Futures.

It uses an ML strategy called Alpha Squeezer v17 — an ensemble of LightGBM classifiers trained with Walk-Forward Optimization (WFO) across 20+ OOS windows. The strategy targets reversal entries on perpetual futures using a Zeno-style 0.5% risk-per-trade position sizing model.

Stack:

Engine_1.py — main orchestration loop (Rich terminal UI, WebSocket streaming, order execution, trade tracking)

alpha_squeezer_v17_strategy.py — feature engineering + prediction pipeline

unified_backtest.py — prep() function used by both backtest and live engine for feature computation

agent5_exact.py / agent6_exact.py — LightGBM model wrappers

config.py — hyperparameter definitions

What I need reviewed

Please do a deep, adversarial code review of this Engine 1 system. Focus on:

1. ARM Signal Logic (Critical)

The armed_str column in the terminal shows -- most of the time. Investigate why macro stays 0 and whether the ARM gate (p_long >= conf_threshold AND macro >= 1) is too strict or misconfigured for live conditions.

The macro column depends on ema_fast - ema_slow > 0.5 * ATR. With candle buffers historically too small (was 120, now 1200), does the EMA convergence work correctly?

Are the confidence thresholds in agent5_configs/ appropriate for live trading vs. backtesting?

2. Trade Execution Safety

Is the order sizing (Zeno 0.5% risk) computed correctly in live conditions? Verify against account_balance fetched from Binance.

Are SL/TP orders placed atomically and correctly? Is there a risk of an orphaned position (entry filled but SL/TP failed)?

Is the active_trades dict properly thread-safe given concurrent WebSocket callbacks?

3. Trade Tracker & PnL Accuracy

Engine1TradeTracker loads from Engine_1/Engine_1_trade_logs.json. Verify the PnL calculation: pnl_usd = (exit_price - entry_price) * qty for LONG and inverted for SHORT. Is this correct for USDT-margined perpetuals?

Is current_capital updated correctly after each close? Can it drift from reality over sessions?

Is the atomic write (tmp → os.replace) implemented correctly on Windows?

4. Rich UI Rendering

The Terminal uses rich.live Live with a 2-second refresh. Is there any risk of deadlock or missed updates from WebSocket callbacks writing to shared state while the Rich thread reads it?

The bottom panel shows: Initial Capital, Current Capital, Total PnL, No. of Trades, Winrate. Are these all computed correctly from Engine1TradeTracker?

5. Macro & Feature Engineering

In unified_backtest.py, prep() computes macro_score via EMA crossover. Does this work correctly when the DataFrame is short (< 800 rows)? Is there a risk of NaN propagating into macro?

Are there any feature leakage risks in how compute_features() is called on the rolling candle buffer?

6. WebSocket Reliability

How does the engine handle WebSocket disconnections mid-trade? Is there a reconnect mechanism?

If a kline callback arrives while an order is being placed, can it trigger a duplicate entry?

7. Overall Architecture Assessment

Is the single-file design of Engine_1.py (~1,900 lines) sustainable? What are the highest-risk coupling points?

What are the top 3 bugs most likely to cause real money loss in the next 24 hours of live trading?

Files to Attach

Please attach these 6 files in order:

Engine_1/Engine_1.py — main engine (~1,900 lines)

Engine_1/alpha_squeezer_v17/alpha_squeezer_v17_strategy.py — strategy + feature pipeline

Engine_1/alpha_squeezer_v17/unified_backtest.py — prep() function used live

Engine_1/alpha_squeezer_v17/agent5_exact.py — LightGBM model wrapper (LONG predictor)

Engine_1/alpha_squeezer_v17/agent6_exact.py — LightGBM model wrapper (SHORT predictor)

Engine_1/alpha_squeezer_v17/config.py — hyperparameter config

Do NOT add the models/ directory (binary .pkl files) or Seeding/ (Parquet data). Only the 6 Python files above.

Output Format Expected

Please provide your review as:

Severity-ranked bug lists, CRITICAL / HIGH / MEDIUM / LOW with exact file + line references

Code fix diffs for each CRITICAL and HIGH issue

Architecture verdict: is this engine production-safe for a \$5,000 live account, and what is the single biggest risk?

Take your time. Accuracy over speed. Real money is on the line.

I've done a careful read of the code you actually pasted. Before diving in, one important thing up front, because it materially changes my review and it's about real money.

Your Engine_1.py has your CoinGlass credentials hardcoded in plaintext:

email = os.environ.get("COINGLASS_EMAIL", "singhkarani0248@gmail.com")
password = os.environ.get("COINGLASS_PASSWORD", "Lu\$er2hero")

Copy

That password is now in your source, in this chat, and almost certainly in your git history. Treat it as compromised. Rotate it immediately, and if you reuse it anywhere else (email, exchange, etc.), rotate those too. Since your CoinGlass login may be tied to accounts that touch money, this is the single most urgent item in the whole review — more urgent than any trading bug. Move both to environment variables with no default.

Now, one honest caveat about scope. You asked me to review order execution, WebSocket reliability, atomic SL/TP placement, and reconnect logic — but three of the six files you described don't match what's in front of me, and the execution logic you're asking me to review isn't in the code that's actually been reviewed.

in front of me, and the execution layer you're asking about isn't in the pasted code at all. I'll be explicit about what I can and cannot verify so you don't get false confidence.

What I can actually see vs. what you described

The code you pasted is: `config.py` (the STRATEGY, `ENGINE` map + `get_param`), `agent5_exact.py` (an Optuna *training* optimizer — not a "LONG predictor wrapper"), the `unified_backtest.py` override shim with `custom_prep`, the `alpha_squeezer_v17_strategy.py` Engine-3 strategy class, and `Engine_1.py`.

Critically:

- There is **no** `agent6_exact.py` in what you pasted, so I can't review the "SHORT predictor."
- **Engine_1.py does no Binance order execution.** It's a scraper + a *paper* trade tracker. `Engine1TradeTracker.trigger_entry` just writes to a JSON file and computes simulated PnL. There is no `account_balance` fetch, no order placement, no SL/TP orders on the exchange, no WebSocket kline stream. So the entire "Trade Execution Safety" and "WebSocket Reliability" sections you asked about cannot be reviewed — that code isn't here. If you have a separate execution module, I need it. If you don't, then this engine is not actually trading, and any belief that it is placing atomic SL/TP orders is itself a dangerous misconception.

I'll review what exists thoroughly, then tell you plainly where the gaps are.

Severity-ranked bug list

CRITICAL

C1 — Hardcoded credentials (`Engine_1.py`, in `main()`). Covered above. Also `singhkaranbir0248@gmail.com` is exposed. Rotate + move to env with no fallback.

C2 — The macro filter in the strategy file and the live engine disagree, and one of them is silently killing all signals.

In `alpha_squeezer_v17_strategy.py`, `compute_features`:

```
df['ema_slow'] = df['Close'].ewm(span=800, min_periods=1).mean()
df['macro'] = np.where(df['Close'] > df['ema_slow'], 1, -1)
```

Here `macro` is `Close > EMA800`, which is *never 0*. It's always ± 1 .

But in `custom_prep` (the `unified_backtest.py` shim that `Engine_1` actually uses via `compute_features`):

```
df['macro_score'] = (df['ema_fast'] - df['ema_slow']) / df['atr'].replace(0, 1e-10)
df['macro'] = np.where(df['macro_score'] > 0.5, 1, np.where(df['macro_score'] < -0.5, -1, 0))
```

Here `macro` is 0 whenever `|ema_fast - ema_slow| < 0.5 * ATR` — i.e., whenever the 200/800 EMAs are within half an ATR of each other. **This is your -- problem.** Your ARIM gate is `p >= conf AND macro >= 1 (long) / macro <= -1 (short)`. In ranging conditions the EMAs sit inside the ± 0.5 ATR band and `macro` is pinned at 0, so no signal can ever fire regardless of model confidence.

Worse: with only ~20 candles of live history (see C3), `ema_fast` (span 200) and `ema_slow` (span 800) with `min_periods=1` are both essentially just the running mean of the same tiny window, so they're nearly *identical*, `macro_score` = 0, and `macro` is **structurally locked at 0**. That fully explains "-- most of the time." It's not that the gate is "too strict" — it's that the feature can't be computed meaningfully on the buffer you're feeding it.

C3 — The live predictor runs on ~20 candles but the features require 800.

In `LiveStrategyPredictor.on_tick_update`:

```
if len(history) < 20:
    return snap
```

Then `compute_features` → `custom_prep` computes EMA-800, `zscore(atr, 100)`, `rolling(100) z-scores` for OI/LS, `ema(span=800)`, etc. With `min_periods=1` these don't error — they silently produce garbage. An EMA-800 seeded from 20 points has essentially no memory of a trend. Every `macro/regime/vol` feature is meaningless until you have hundreds of bars. You are generating predictions on features that do not resemble anything the model was trained on. Even if you fixed the ARIM gate, the model inputs are invalid. Raise the gate to at least ~800 (ideally 900–1000 to let EMA-800 stabilize), and gate `min_periods` accordingly.

Note the strategy file itself uses `DEFAULT_KLINE_LIMIT = 1000` and warns "Warming up" until 1000 bars — so the *strategy* class knows it needs 1000, but the `LiveStrategyPredictor` in `Engine_1.py` (a completely separate, parallel implementation) only requires 20. These two code paths have drifted apart.

C4 — There is no order execution, yet the UI implies live trading. As noted,

`trigger_entry/check_exits` are pure simulation. `check_exits` compares `current_price` (a scraped CoinGlass legend value, polled at 2 Hz off a browser DOM) against SL/TP. This means: (a) no real position exists, and (b) even the simulated fills use a laggy, DOM-scraped price that can miss wicks entirely between polls, so simulated SL/TP fills are optimistic and unrealistic. If you believe this engine is protecting a \$5,000 account with real stops, it is not.

HIGH

H1 — TICK_SIZES.get(s, 0.01) fallback silently corrupts the footprint POC for unknown symbols.

A default tick of 0.01 on, say, DOGE (real tick ~0.00002) buckets everything into one price level, making POC meaningless. Fail loudly instead of defaulting. 🚨

H2 — parse_float mis-parses hex-free but ambiguous suffixes and mangles funding. In `_get_cg_series` (strategy file) you do `str.replace('M', 'e')` etc., but `parse_float` in `Engine_1` handles suffixes differently (K/M/B/T multipliers). Two different parsers for the same CoinGlass strings = guaranteed inconsistency between what's seeded to Excel and what's used live. Also, funding in the JS shim is multiplied by 100 and given a %, then `parse_float` strips % and does *not* divide back — so funding is off by 100x relative to the model's training scale unless the training parquet used the same convention. Verify the funding units end-to-end; a 100x feature error will wreck any funding-sensitive prediction.

H3 — Feature/label alignment and potential leakage in the training loop (`agent5_exact.py`). `build_labels_fast` is computed on the full `df`, then `train/val` masks are applied. The `PURGE_GAP = 48` gap is applied between `val_start` and `train_end`, which is good, but labels look forward `MAX_BARS`; you purge 48 bars but if `MAX_BARS > 48` the tail of training can still peek into the validation window. Confirm `PURGE_GAP >= MAX_BARS` (or `MAX_BARS + horizon`). If `MAX_BARS = 96` as in config, a 48-bar purge is **half** what's needed — direct label leakage into the walk-forward score, which inflates every Optuna `best_value` and gives you falsely confident configs.

H4 — current_capital drift across sessions (`Engine1TradeTracker.load_history`). On load you recompute `current_capital = initial + sum(pnl_usd)`, but active (unclosed) trades are restored *without* re-validating their entry against current market — and their eventual close appends to history, so capital is only ever correct if every active trade eventually closes through `check_exits`. If the process dies with an open simulated trade and you restart, the open trade's mark-to-market is lost and re-entered at stale SL/TP. Over many sessions with crashes, capital can drift. For paper trading this is cosmetic; if you ever wire real execution to this tracker, it's a real accounting bug.

H5 — Two divergent feature pipelines is the core architectural hazard. `custom_prep` (used by `Engine_1`) selects `ga_selected` = `[ 'z_cvd_4', 'z_btc_4', 'z_btc_10', 'z_cvd_20', 'z_btc_20', 'btc_cvd_mom', 'z_oi', 'z_ls', 'liq_long_5' ]` and returns `feats` = `[ f for f in ga_selected if f in feats ] + existing_new_feats`. But `feats` at that point only contains the base `feats` list, and macro was appended to `feats` earlier then never re-added — so macro is computed as a column but **dropped from the returned feature list**, while the model in `on_tick_update` calls `self.models_long[symbol].feature_name()` to select columns. If the model was trained expecting macro (or the enhancement `feats`) and `custom_prep` doesn't produce them, `last_row[expected_feats]` throws a `KeyError`, caught by the broad `except Exception` in `on_tick_update`, printed once, and **silently swallowed forever**. That broad `except` is hiding exactly the bug you're chasing. Add the failing feature name to the log.

MEDIUM

M1 — Broad `except Exception`: blocks everywhere swallow the diagnostics you need. `on_tick_update`, `objective`, `optimize_asset_route_payload`, the response handler. At minimum log `traceback.format_exc()`. Right now the engine can be failing every single prediction and you'd see one line of output.

M2 — `_maybe_reload_models` (strategy file) has a no-op locked block — it acquires the lock, passes, releases, then re-checks unlocked. The TOCTOU it claims to prevent is not prevented. Harmless but misleading.

M3 — `MAX_DD_LIMIT`, `ZENO_DENOM`, `RISK_CAP` are redefined after import in `agent5_exact.py` (`MAX_DD_LIMIT` = 300.0 right after importing it from `unified_backtest`). This shadows the compiled value and the strategy file uses `ZENO_DENOM` = 4.0 while `agent5` uses 3.5999999999999996. Inconsistent risk math between `backtest` and (intended) live sizing.

M4 — `resample('15min')` on scraped `CoinGlass` history assumes clean timestamps. `_get_cg_series` builds a `DatetimeIndex` from feed tuples; if any timestamp is duplicated or out of order (very likely from 2 Hz DOM scraping), `.last()` silently picks arbitrary values.

M5 — Rich Live + shared store: the read is described as "GIL-atomic shallow copy," which is *mostly* true for `dict(self._data)`, but the `async` updates use `asyncio.Lock` while the `renderer` runs in the same event loop — so no true data race, but partial-field snapshots are possible (`renderer` sees a `snap` mid-update between fields). Cosmetic only.

LOW

- `warnings.filterwarnings('ignore')` globally hides `pandas/sklearn` correctness warnings.
- `tick_size` = `diffs[diffs > 0].min()` in `objective` can pick up sub-tick float noise as the "tick."
- `os.replace` atomic write is correct on Windows (good — this one you got right).
- The `xbg ... device="cuda"` in `agent5_exact.py` will crash on a CPU-only box; the `except Exception: return -99999.0` will then make every trial return the failure sentinel and Optuna will "succeed" with a garbage best. Log it.

Code fixes for CRITICAL/HIGH

Credentials (C1):

```
email = os.environ["COINGLASS_EMAIL"]
password = os.environ["COINGLASS_PASSWORD"]
```

No defaults. Rotate the leaked password now.

Macro/warmup (C2, C3) — in `LiveStrategyPredictor.on_tick_update`:

```
# was: if len(history) < 20:
if len(history) < 850:      # EMA-800 needs real history
    return snap
```

And unify the macro definition. Pick one. If the model was trained with `custom_prep`'s macro (the ±0.5 ATR band version), then the strategy file's `Close > ema_slow` version is wrong and must be replaced with:

```
df['ema_fast'] = df['Close'].ewm(span=200, min_periods=200).mean()
df['ema_slow'] = df['Close'].ewm(span=800, min_periods=800).mean()
macro_score = (df['ema_fast'] - df['ema_slow']) / df['atr'].replace(0, 1e-10)
df['macro'] = np.where(macro_score > 0.5, 1, np.where(macro_score < -0.5, -1, 0))
```

Setting `min_periods` equal to span forces NaN (and thus no signal) until enough data exists, which is *correct* — better to arm nothing than to arm on garbage.

Surface the swallowed error (H5, M1) — in `on_tick_update`:

```
except Exception as e:
    import traceback
    print(f"[Strategy] {symbol} prediction error: {e}\n(traceback.format_exc())")
```

Run once and you'll almost certainly see a `KeyError` naming the exact feature `custom_prep` fails to produce. That's the real root cause of --, and I can't name the specific column without seeing the training feature list / the model file's `feature_name()`.

Purge gap (H3) — in `agent5_exact.py`:

```
PURGE_GAP = 96      # must be >= MAX_BARS (label horizon), was 48
```

Tick size fallback (H1):

```
tick = TICK_SIZES.get(s)
if tick is None:
    raise KeyError(f"No TICK_SIZE configured for {s}; refusing to guess POC bucket.")
```

Architecture verdict

Is this production-safe for a \$5,000 live account? **No** — and not for the reasons in your question, but for a bigger one: the code you shared does not place live orders at all. It's a browser scraper feeding a paper-trade simulator. So either (a) real execution lives in a module you didn't share, in which case I can't certify safety and need that file, or (b) it doesn't exist, in which case the "\$5,000 live account" is not actually being traded by this code.

The single biggest risk, if you do wire real execution to this design as-is: **you have two divergent feature pipelines and a macro feature that is structurally stuck at 0 on live buffers, so the model is either never firing or firing on inputs that don't match training — and every failure is hidden by broad `except` blocks.** You would be trading real money on a model whose live inputs you cannot verify match its training distribution. That's the "unknowingly losing money" scenario.

Top 3 things most likely to cause loss in the next 24h (assuming execution gets connected): the EMA-800-

on-20-bars invalid-feature problem (C2/C3); the insufficient purge gap inflating your config confidence so you deploy over-fit thresholds (H3), and the silent exception swallowing that means you won't notice either until the account is down (M1/H5).

Concrete next step: paste `agent6_exact.py`, the real order-execution module (if it exists), and the output of one live run after adding the traceback print. With those three I can give you the execution-safety, atomic-SL/TP, and WebSocket-reconnect review you actually asked for — right now that code simply isn't in front of me, and I won't guess about money-moving logic I can't see.

And again: rotate that password before you do anything else.

📄 👍 💬 ...

🔒 AI Chat

• Video+PPT+image AI — pay once, not 3x →

Ask anything, create anything



Claude Opus 4.8

